To parse **HL7 (likely HL7 v2.x)** similar to how MuleSoft handles it (structured parsing, transformation, validation), the equivalent library depends on your programming language.
Below are the **most widely used HL7 parsing libraries by ecosystem**:

---

🟦 **Java (Closest to MuleSoft ecosystem)**
✅ **HAPI HL7 (Best Equivalent)**
**Library:** HAPI (HL7 API)
Website: https://hapifhir.io (covers both HL7 v2 and FHIR)
For HL7 v2:

- <dependency>
- <groupId>ca.uhn.hapi</groupId>
- <artifactId>hapi-base</artifactId>
- </dependency>

**Why HAPI?**

- Full HL7 v2 parser
- Message validation
- Segment-level access
- Message building
- Encoding/decoding
- Widely used in healthcare integrations

**Example:**

- Parser parser = new PipeParser();
- Message message = parser.parse(hl7String);
-
- PID pid = (PID) message.get("PID");
- String patientId = pid.getPatientIdentifierList(0).getIDNumber().getValue();

👉 If you want MuleSoft-like behavior in Java → **HAPI is the direct equivalent**.

---

🟦 **.NET (C#)**
✅ **NHapi (Port of HAPI)**
GitHub: https://github.com/duaneedwards/nHapi

- var parser = new PipeParser();
- var message = parser.Parse(hl7String);
-
- var pid = message.GetStructure("PID") as PID;
- string patientId = pid.PatientIdentifierList[0].IDNumber.Value;

Equivalent to HAPI but for .NET.

---

🟦 **Python**
✅ **hl7apy (Best Structured Option)**

- pip install hl7apy

Example:

- from hl7apy.parser import parse_message
-
- msg = parse_message(hl7_string)
- patient_id = msg.PID.patient_identifier_list[0].id_number.value

Features:

- Structured access
- Validation
- Message creation
- Encoding

Another lighter option:

- hl7 package (less structured, more raw parsing)

---

🟦 **Node.js**

✅ **node-hl7**

- npm install node-hl7

Example:

- const hl7 = require('node-hl7');
- 
- let parser = new hl7.Parser();
- let msg = parser.parse(hl7String);
- 
- let pid = msg.get('PID');

For more enterprise-grade:

- simple-hl7
- hl7-standard

But Node HL7 libraries are not as mature as Java/.NET.

---

🟦 **Go**

✅ **github.com/kardianos/hl7**

Lightweight parsing

---

🟦 **If You Also Need FHIR**

If you're working with **FHIR instead of HL7 v2**, then:

- Java → HAPI FHIR
- .NET → Firely .NET SDK
- Python → fhir.resources

---

🔥 **If You Want MuleSoft-Like Capabilities**

MuleSoft HL7 module typically provides:

- Parsing
- Transformation
- Validation
- Mapping to JSON/XML
- Routing

To replicate that fully, you'll typically combine:

| Need | Tool |
|---|---|
| HL7 parsing | HAPI / NHapi / hl7apy |
| Transformation | MapStruct / AutoMapper / custom mapping |
| Validation | Built-in HL7 validation |
| Routing | Spring Boot / ASP.NET / FastAPI |

---

💡 **Recommendation by Use Case**

| Scenario | Best Choice |
|---|---|
| Enterprise Java | HAPI HL7 |
| Enterprise .NET | NHapi |
| Python microservice | hl7apy |
| Integration-heavy system | Java + HAPI |
| Lightweight script | Python hl7 |

---

If you tell me:

- What language?
- HL7 v2 or FHIR?
- Just parsing or full transformation pipeline?

I can give you a concrete architecture example similar to MuleSoft's flow.

Great question — this is exactly the right way to think about a MuleSoft → Python/AWS migration.
When building a **utility that converts MuleSoft flows to Python-based AWS code**, you need to understand what MuleSoft actually provides under the hood and map each component to AWS + Python equivalents.
Below is a structured breakdown.

---

## 1️⃣ Core MuleSoft Components You Must Cover

### A. Flows & Subflows

**Mule Concept:**

- <flow>
- <sub-flow>
- <choice>
- <scatter-gather>
- <async>

**AWS/Python Equivalent:**

| Mule | AWS + Python |
|---|---|
| Flow | Lambda / FastAPI route / Step Function |
| Subflow | Python function / module |
| Choice | if/else |
| Scatter-Gather | asyncio / Step Functions parallel state |
| Async | Lambda async invoke / SQS |

Your converter must:

- Parse flow structure
- Convert to Python execution graph
- Handle sync vs async correctly

---

### B. Connectors

Mule has many connectors:

- HTTP
- DB
- JMS
- SFTP
- Salesforce
- HL7
- SAP
- etc.

**AWS Mapping Examples:**

| Mule Connector | Python + AWS Equivalent |
|---|---|
| HTTP Listener | API Gateway + Lambda |
| HTTP Request | requests / httpx |
| DB | SQLAlchemy |
| JMS | SQS / SNS |
| File/SFTP | S3 + Paramiko |
| Scheduler | EventBridge |
| VM Queue | SQS |
| ObjectStore | DynamoDB / Redis |
| Salesforce | simple-salesforce SDK |

Your utility must:

- Detect connector type
- Generate correct AWS SDK or Python library
- Generate IAM policy hints (optional but powerful)

---

**C. DataWeave (Critical)**
This is the hardest part.
**Mule:**

- %dw 2.0
- output application/json
- ---
- {
- id: payload.userId,
- name: payload.firstName ++ " " ++ payload.lastName
- }

**Python Equivalent:**

- Dictionary transformation
- List comprehension
- JSON mapping
- Possibly Pydantic models

You must build:

- DataWeave AST parser
- Translator to Python expressions
- Mapping for:
  - map
  - filter
  - reduce
  - pluck
  - flatten
  - default
  - null handling

DataWeave → Python mapping is 40–50% of the migration complexity.

---

**D. Error Handling**
**Mule:**

- <error-handler>
- <on-error-propagate>
- <on-error-continue>

**Python Equivalent:**

- try:
- ...
- except SpecificError:
- ...

Or in AWS:

- Lambda error handling
- Step Function catch blocks

Your tool must:

- Parse error types
- Convert to try/except or Step Function error states

---

**E. Properties & Configuration**
**Mule:**

- mule-artifact.json
- application.yaml
- secure properties

**AWS Equivalent:**

- SSM Parameter Store
- Secrets Manager
- environment variables
- config.py

Your converter must:

- Extract properties

- Replace with os.environ.get(...)
- Optionally generate Terraform/CloudFormation hints

---

**F. Batch Processing**
**Mule:**
<batch:job>
**AWS Equivalent:**
- Step Functions
- SQS + Lambda
- AWS Batch

Batch requires architectural transformation, not 1:1 code translation.

---

**G. Logging**
**Mule:**
<logger>
**Python:**
- import logging
- logger = logging.getLogger(__name__)

---

**H. Transaction Management**
**Mule:**
Transactional scopes
**AWS:**
- RDS transactions
- Step Functions orchestration
- No distributed XA (usually re-architected)

---

## 2️⃣ Architecture-Level Mapping

MuleSoft Runtime → AWS Equivalent

| Mule Runtime | AWS |
|---|---|
| Mule App | Lambda / ECS |
| API Manager | API Gateway |
| Anypoint MQ | SQS |
| Object Store | DynamoDB |
| Scheduler | EventBridge |
| Runtime Clusters | Lambda + autoscaling |

---

## 3️⃣ What Your Conversion Utility Must Handle

**Step 1: Parse Mule XML**
You must parse:
- flows
- processors
- connectors
- DataWeave scripts
- error handlers

Use:
- lxml
- xml.etree.ElementTree

Build an internal AST.

---

**Step 2: Build Intermediate Representation (IR)**
Do NOT directly convert XML → Python.
Instead:
Mule XML → Internal Flow Graph → Python Code Generator
This makes your tool scalable and testable.

---

**Step 3: Generate AWS-Aware Python**
You likely need templates for:

- Lambda handler
- FastAPI app
- SQS consumer
- Step Function JSON
- IAM policy stub

Use:

- Jinja2 templates
- Code generation patterns

---

**4️⃣ Major Challenges**
🚨 **DataWeave Translation**
Hardest part.
🚨 **Async vs Sync semantics**
Mule async doesn't map 1:1 to Lambda.
🚨 **Transactions**
Mule supports more enterprise-style transaction management.
🚨 **Connectors with no AWS equivalent**
May require SDK calls.

---

**5️⃣ Recommended Target Architecture**
Instead of raw Python scripts, generate:
Option A (Most common):

- API Gateway
- Lambda (Python)
- DynamoDB / RDS
- SQS
- EventBridge

Option B (More complex):

- Step Functions for orchestration

---

**6️⃣ Suggested Tool Architecture**

- mule_converter/
- │
- ├── xml_parser.py
- ├── flow_graph_builder.py
- ├── dataweave_parser.py
- ├── aws_mapper.py
- ├── python_generator.py
- ├── templates/
- │ ├── lambda.j2
- │ ├── sqs_consumer.j2
- │ └── step_function.j2

---

**7️⃣ Migration Scope Checklist**
When building your converter, ensure you cover:

- Flows & subflows
- Choice routers
- Scatter-gather
- Connectors (HTTP, DB, MQ, File)
- DataWeave transformations
- Error handlers
- Properties
- Batch jobs

- Logging
- Transactions (if needed)
- Security (TLS, OAuth)

---

🔥 **Important Reality**
MuleSoft → AWS is not pure code translation.
It's **runtime-to-cloud architecture migration**.
Your tool must:

- Translate logic
- Re-architect infrastructure
- Generate deployment artifacts

---

If you'd like, I can next provide:

- A visual mapping diagram
- A minimal working prototype structure
- Or a phased roadmap to build this converter properly

Which stage are you in — design, PoC, or full migration product?

Excellent — this is now a **product design problem**, not just a code problem.
You're building a **MuleSoft → Python (AWS-native) automated migration utility**.
Below is a **complete design + PoC execution roadmap**, structured like something you could actually implement.

---

## 🎯 1. Define Scope Clearly (Critical)

Before design, define PoC boundaries.
For Phase 1 (realistic PoC), support only:

### ✅ Supported Mule Features

- HTTP Listener
- HTTP Request
- DB Connector
- Logger
- Choice Router
- Simple DataWeave (mapping, map, filter, default)
- Error handler (basic)
- Properties file

### ❌ Explicitly Exclude (Phase 2)

- Batch jobs
- Scatter-Gather
- JMS
- Complex DataWeave functions
- Transactions
- Custom connectors

Without scope control, the project explodes.

---

## 🏗️ 2. High-Level Architecture

Do NOT directly convert XML → Python.
Instead use a **3-layer compiler architecture**:

- Mule XML
- ↓
- Parser Layer
- ↓
- Intermediate Representation (IR)
- ↓
- Transformation Engine
- ↓
- AWS + Python Code Generator
- ↓
- Project Scaffold Output

This makes it maintainable and scalable.

---

## 🧠 3. Core Design Components

### 3.1 Mule XML Parser

Input:

- Mule XML file

Output:

- Internal representation of flows

Use:

- lxml

Extract:

- flows
- processors
- connectors
- error handlers
- DataWeave blocks

---

### 3.2 Intermediate Representation (IR)

This is the most important design decision.
Example IR:

- class Flow:
- name: str
- trigger: Trigger
- steps: list[Processor]
- error_handler: ErrorHandler
-
- class HTTPListenerTrigger:
- path: str
- method: str
-
- class DBConnector:
- query: str
-
- class Choice:
- conditions: list[ConditionBlock]
-
- class Transform:
- dataweave_ast: object

The IR must be:

- Platform-agnostic
- Independent of Mule
- Independent of AWS

Think of it as your "compiler AST".

---

### 3.3 DataWeave Parser (Hardest Part)

For PoC:

- Do NOT build full DW parser.
- Convert only simple expressions using pattern translation.

Example support:

| DataWeave | Python |
|---|---|
| payload.id | payload["id"] |
| ++ | + |
| default | or or if None |
| map | list comprehension |
| filter | list comprehension |

Phase 1 Strategy:

- Use regex-based transformation
- OR build simple grammar with lark

Example:

- def translate_dw(expr: str) -> str:
- expr = expr.replace("payload.", 'payload["')
- ...

Keep it simple for PoC.

---

### 3.4 AWS Mapper Layer

Map IR components to AWS architecture.
Example:

- HTTP Listener → API Gateway + Lambda
- DB Connector → SQLAlchemy
- Logger → logging
- Choice → if/else

This layer decides:

- Is this Lambda?
- Is this SQS consumer?
- Is Step Function required?

For PoC:
Always generate:

- Single Lambda app
- API Gateway
- RDS connector

Keep architecture simple.

---

**3.5 Code Generator**
Use:

- Jinja2 templates

Templates:
**lambda_handler.py.j2**

- def handler(event, context):
- payload = event.get("body")
- 
- {{ generated_flow_code }}
- 
- return {
- "statusCode": 200,
- "body": result
- }

**db.py.j2**
**requirements.txt.j2**
Generator fills templates with translated IR logic.

---

📦 **4. Generated Output Structure**
Your tool should generate:

- output_project/
- │
- ├── lambda_handler.py
- ├── services/
- │ └── db.py
- ├── utils/
- │ └── transforms.py
- ├── requirements.txt
- ├── template.yaml (SAM)

Optional:

- Terraform
- CloudFormation
- CDK

---

🛠 **5. PoC Development Plan (8 Weeks)**

---

**Week 1–2: XML Parsing + IR**

- Parse flows
- Build IR objects
- Unit test parser

Deliverable:

- Print parsed flow structure

---

**Week 3–4: DataWeave Basic Translator**
Support:

- Field mapping

- Concatenation
- Simple condition
- map/filter

Deliverable:

- Convert DW snippet to valid Python

---

**Week 5: Code Generator**

- Create Jinja templates
- Generate minimal Lambda
- Deployable via SAM

Deliverable:

- End-to-end generation for 1 simple Mule app

---

**Week 6: Add DB + HTTP request support**

- SQLAlchemy integration
- requests library support

---

**Week 7: Error Handling**

Convert:

- on-error-propagate

to:

- try:
- except:
- raise

---

**Week 8: Testing & Packaging**

- CLI tool
- Logging
- Example migration demo

---

### 🧪 6. PoC Demo Example

Input Mule Flow:

```
<flow name="userFlow">
  <http:listener path="/users" method="GET"/>
  <db:select>SELECT * FROM users</db:select>
  <ee:transform>
    %dw 2.0
    output application/json
    ---
    payload map (u) -> {
      id: u.id,
      name: u.name
    }
  </ee:transform>
</flow>
```

Output:

```
def handler(event, context):
    users = db.select("SELECT * FROM users")

    result = [
        {
            "id": u["id"],
            "name": u["name"]
        }
        for u in users
    ]

```

- return response(result)

---

### 🧩 7. Technical Stack

| Layer | Tool |
|---|---|
| XML Parsing | lxml |
| IR Model | dataclasses |
| DataWeave Parsing | lark or regex |
| Code Templates | Jinja2 |
| CLI | Typer |
| Packaging | Poetry |
| Testing | pytest |

---

### ⚠️ Hard Truths

This will NOT:

- Support full Mule runtime features
- Automatically migrate complex enterprise flows
- Replace human refactoring

But it can:

- Automate 60–70% of simple APIs
- Accelerate migration dramatically

---

### 🧠 8. Future Phase (If Productizing)

- Full DW grammar parser
- Visual flow graph UI
- AI-assisted edge case translation
- CloudFormation/CDK generation
- Cost estimator

---

### 🔥 Strategic Advice

Your biggest leverage is:

👉 Strong IR design
👉 Controlled scope
👉 Template-driven generation

If those are solid, this becomes a real migration platform.

---

If you want, next I can give you:

1. A detailed IR class design
2. A sample folder structure with starter code
3. A minimal working PoC code skeleton
4. A productization roadmap (enterprise-grade tool)

Which direction do you want to go next?

Excellent — now we're talking about building a **deterministic compiler + intelligent assistant**, not a random code generator.
You want:

1. ✅ Where LLM integration is most useful
2. ✅ How to keep output consistent for the same Mule input

These two goals require **very careful placement of LLM usage**.

---

### 🎯 Core Principle

If you want consistent output:
❌ Do NOT let the LLM generate final code freely
✅ Use deterministic compiler logic for structure
✅ Use LLM only where rules become too complex
Think of LLM as a **controlled helper**, not the compiler.

---

### 🏗 Where LLM Is Most Useful in Your Architecture

Here's your architecture again:
Mule XML
↓
Parser
↓
IR (Intermediate Representation)
↓
Transformation
↓
Code Generator
Now let's insert LLM strategically.

---

### 🔥 Best Places to Use LLM

#### 1️⃣ DataWeave → Python Translation (Primary Use Case)

This is the hardest and most dynamic part.
Instead of building full DW grammar parser, you can:

- Extract DW block
- Send DW snippet to LLM
- Ask it to translate to Python expression
- Validate result before accepting

Example prompt:

```
Translate the following DataWeave expression to Python.
Rules:
- Use dict access with ["key"]
- No explanations
- Output only Python expression
- Do not use external libraries

DataWeave:
%dw 2.0
output application/json
---
payload map (u) -> {
  id: u.id,
  name: u.name
}
LLM Output:
[
   {
      "id": u["id"],
      "name": u["name"]
```

```
    }
    for u in payload
]
```

This saves massive engineering effort.

---

### 2️⃣ Unsupported Connector Translation

If your parser finds:

<salesforce:create ...>

You can:

- Ask LLM to suggest Python equivalent
- Or generate boto3 / SDK snippet

LLM is good at:

- Mapping enterprise connectors
- Suggesting SDK usage
- Generating boilerplate

---

### 3️⃣ Complex Conditional Logic

Mule expressions like:

#[payload.user.age > 18 and vars.type == "A"]

Instead of writing complex parser,
LLM can convert to Python safely.

---

### 4️⃣ Migration Report Generator (Very High Value)

Use LLM to:

- Explain migration changes
- Highlight unsupported features
- Produce risk assessment

This is extremely valuable for enterprise adoption.

---

## ❌ Where NOT to Use LLM

To maintain consistent output:
Do NOT use LLM for:

- Project scaffolding
- File structure
- Lambda template generation
- AWS architecture decisions
- Flow control generation
- Logging setup

Those must be deterministic templates.

---

## 🧠 Recommended Hybrid Design

**Deterministic Core**

- XML → IR (pure code)
- IR → AWS mapping (pure rules)
- IR → template filling (pure code)

**LLM-Assisted Modules**

- DW translation
- Complex expression rewrite
- Connector explanation
- Code refactoring suggestions

---

## 🔐 How to Ensure Consistent Output

This is the critical part.

---

### 1️⃣ Use Temperature = 0

Always:

temperature = 0
top_p = 1

---

**2️⃣ Lock Model Version**
Never use:
latest
Always use:
gpt-4.1
or specific version.

---

**3️⃣ Use Strict Prompt Template**
Never dynamically change prompt wording.
Create a fixed template like:
SYSTEM:
You are a deterministic compiler assistant.

RULES:
- Output only valid Python.
- No explanations.
- No comments.
- Use double quotes only.
- Use dict access with ["key"].
- No random variable naming.
- Preserve input variable names exactly.

USER:
<DATAWEAVE_SNIPPET>
Do not alter structure.

---

**4️⃣ Post-Processing Validation Layer**
After LLM returns Python:

- Parse using ast.parse()
- Reject if invalid
- Normalize formatting with Black
- Remove trailing spaces
- Standardize indentation

---

**5️⃣ AST Normalization (Advanced but Powerful)**
To ensure same logic always generates same formatting:

- Convert LLM output to AST
- Regenerate code from AST
- That guarantees consistent formatting

This removes minor token differences.

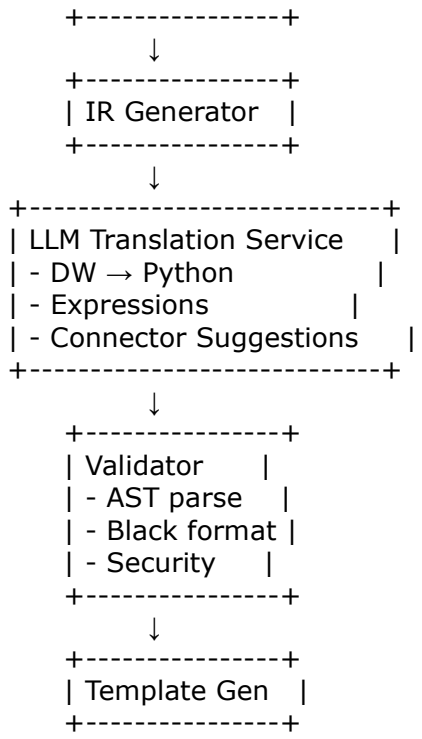
---

**6️⃣ Caching for Absolute Consistency**
For production-grade determinism:
Hash the DW block:
hash = sha256(dw_snippet)
Store translation result.
If same DW appears again → reuse cached Python.
Now output is 100% consistent.

---

🏆 **Best Architecture for Your Goal**

```
+----------------+
| Deterministic  |
| Mule Parser    |
```




---

### 🧪 PoC-Level LLM Integration Plan

Phase 1:

- Only use LLM for DataWeave block translation
- Everything else deterministic

Phase 2:

- Add expression conversion
- Add migration report generation

Phase 3:

- Add fallback reasoning for unsupported features

---

### ⚠️ Important Warning

If you let LLM:

- Generate entire Lambda
- Design AWS architecture
- Create file structure

You lose reproducibility.
Your tool becomes unstable.

---

### 🔥 Final Recommendation

For your converter:
LLM role = "Controlled Expression Translator"
NOT
LLM role = "Code Generator"
That is the difference between:

- A research toy
- A production migration tool

---

If you'd like, next I can give you:

1. A production-grade LLM prompt template for DW translation
2. A deterministic validation pipeline example
3. A caching strategy design
4. A hybrid rule-based + LLM fallback architecture

Which one would you like to go deeper into?

Excellent — this is the **most critical piece** of making your Mule → Python converter stable.

Below is a **production-grade, deterministic prompt template** specifically designed for:

- 🔒 Consistent output
- 🔁 Repeatable results
- 🧠 Controlled translation
- 🚫 No hallucinations
- ✅ Python-safe output

This assumes:

- temperature = 0
- Fixed model version
- You validate output via ast.parse

---

### 🏗 Design Goals of This Prompt

1. Output only valid Python
2. No explanations
3. No extra imports
4. No formatting variation
5. Deterministic variable naming
6. Preserve input variable names
7. Enforce dictionary access style
8. Avoid creative reinterpretation

---

### 🔐 SYSTEM PROMPT (Never Change This)

Use this as your **system message**:

- You are a deterministic compiler assistant.
-
- Your task is to translate DataWeave 2.0 expressions into pure Python expressions.
-
- STRICT RULES:
- - Output ONLY valid Python code.
- - Output a single Python expression unless explicitly told otherwise.

- - No explanations.
- - No comments.
- - No markdown.
- - No backticks.
- - No extra text.
- - No imports.
- - Do not wrap output in functions.
- - Do not add try/except.
- - Do not introduce new variables unless required by the transformation.
- - Preserve original variable names exactly as given.
- - Use double quotes only.
- - Use dictionary access style: obj["field"] instead of obj.field.
- - Never use getattr.
- - Never use external libraries.
- - Use Python list comprehensions for map/filter.
- - Use conditional expressions for inline conditionals.
- - If translation is not possible, output exactly: TRANSLATION_ERROR

Do NOT modify this system prompt once deployed.

---

### 📜 USER PROMPT TEMPLATE (Stable Template)

Use a fixed template like this:

- Translate the following DataWeave expression to Python.
-
- CONTEXT:
- - The variable "payload" is a Python dict or list.
- - The variable "vars" is a Python dict.
- - The variable "attributes" is a Python dict.
- - Null in DataWeave must be translated to None.
- - "default" operator must be translated using Python conditional logic.
-
- DATAWEAVE:
- <INSERT_DATAWEAVE_HERE>

Never change wording order.

Only substitute <INSERT_DATAWEAVE_HERE>.

---

### 🧠 Example 1 — Simple Mapping

**Input DW**

```
%dw 2.0
output application/json
---
{
  id: payload.userId,
  name: payload.firstName ++ " " ++ payload.lastName
}
```

**LLM Output (Expected)**

```
{
    "id": payload["userId"],
    "name": payload["firstName"] + " " + payload["lastName"]
}
```

---

### 🧠 Example 2 — map

**Input DW**

```
%dw 2.0
output application/json
---
payload.users map (u) -> {
  id: u.id,
  active: u.active default false
}
```

**Output**

```
[
    {
        "id": u["id"],
        "active": u["active"] if u["active"] is not None else False
    }
    for u in payload["users"]
]
```

---

### 🧠 Example 3 — filter

**Input**

- payload.users filter (u) -> u.age > 18

**Output**

- [
- u
- for u in payload["users"]
- if u["age"] > 18
- ]

---

### 🛡 Post-Processing Layer (Mandatory)

Even with temperature 0, enforce this:

**Step 1: AST Validation**

- import ast
-
- try:
- ast.parse(output)
- except SyntaxError:
- raise TranslationError

---

**Step 2: Normalize Formatting**

Run:

- black --line-length 88

This guarantees consistent formatting across runs.

---

**Step 3: Security Check**

Reject output if it contains:

- import
- exec
- eval
- __
- open(
- os.

---

### 🔁 For Absolute Consistency

Add caching:

- import hashlib
- 
- key = hashlib.sha256(dw_snippet.encode()).hexdigest()
- 
- if key in cache:
- return cache[key]
- 
- result = call_llm(...)
- cache[key] = result

Now identical DW always returns identical Python.

---

### 🧩 Optional: Add Guardrails Section

You can extend the system prompt with strict operator mappings:

- OPERATOR MAPPING RULES:
- - ++ → +
- - default → conditional expression checking for None
- - map → list comprehension
- - filter → list comprehension with if
- - and/or/not remain the same
- - null → None

This reduces ambiguity.

---

### 🚨 Critical Production Advice

Do NOT:

- Let the LLM decide architecture
- Let the LLM create full files
- Let the LLM name new helper functions

Keep it expression-only.

---

### 🏆 Final Production Setup

Use:

- Model: fixed version (e.g., gpt-4.1)

- temperature: 0
- top_p: 1
- max_tokens: tight but sufficient
- system prompt: immutable
- user prompt: templated
- AST validation
- Black formatting
- Hash-based caching

This gives you:

✓ Stable
✓ Deterministic
✓ Production-safe
✓ Repeatable

---

If you'd like, next I can give you:

- A fallback strategy when LLM returns TRANSLATION_ERROR
- A hybrid rule-based preprocessor to reduce LLM calls by 60%
- A full Python service wrapper for the translation engine
- A regression testing framework for translation validation

Which direction do you want to go?